Problem statements – Tracks and Domain

PS	Title	Domain	Core challenge	Dataset
Track 1 · Agentic AI & Intelligent Automation				
1.1	Code-Mixed Support Triage Assistant	Customer service	Must triage Tanglish/Hinglish messages the team writes as a test set, and escalate to a human with a reason when unsure	Bitext Customer Support (Hugging Face)
1.2	Ask-Your-Data Sales Agent (text-to-SQL)	Retail	Must pass "trap" questions on cancelled invoices, returns and negative quantities, show its SQL, and ask back when a question is ambiguous	UCI Online Retail II
1.3	Maintenance Copilot: agentic RAG over logbooks	Aviation maintenance	Noisy, abbreviation-heavy logbook text; every answer cites the source records	MaintNet aviation logbook
Track 2 · Smart Manufacturing & Computer Vision				
2.1	Maintenance Shift Planner	Industrial	Prediction is only the input: alert threshold set from a cost model, then a fixed number of technician slots per shift assigned to machines	AI4I 2020, UCI
2.2	Leaf Doctor for Real Field Photos	Agriculture	Field photos instead of lab images; the app rejects poor photos, says "not sure" below a confidence bar, and gives retake advice	PlantDoc (GitHub)
2.3	Unknown-Defect Steel Inspector	Manufacturing QA	One defect class is held out of training; the system must flag it as "unknown" rather than mislabel it, then learn it from 5 operator-labelled examples	NEU Surface Defect Database (NEU-CLS)
Track 3 · Sustainable Industry & Energy Intelligence				
3.1	Safe Outdoor Hours Planner for Schools	Environment	Not AQI prediction: recommends the safest hours for outdoor activity and refuses to advise when sensor data is missing or stale	Air Quality Data in India (Kaggle)
3.2	Workplace EV Charging Scheduler	Clean mobility	Replay real charging sessions and keep the site under a power cap while meeting drivers' energy needs by departure; compare against uncontrolled charging	Caltech ACN-Data (free sign-up)

PS	Title	Domain	Core challenge	Dataset
3.3	BIM Model Checker (IFC)	Construction	Team writes its own rule set (e.g. doors narrower than 900 mm, unnamed spaces) and reports violations with element IDs	buildingSMART IFC sample files
Track 4 · AI for Supply Chain & Logistics				
4.1	Markdown Before Waste: near-expiry pricing	Grocery retail	Team designs its own expiry and demand simulator and justifies each assumption	Synthetic, generated by the team
4.2	Warehouse Digital Twin: 3D Picking Floor Game	Warehousing / digital twin	A browser 3D twin of a picking floor driven by real order lines; the player re-slots products and the twin replays the same orders to score walking distance and pick time before vs after	UCI Online Retail II (one month of orders)
4.3	Last-Mile Route Planner with Live Re-planning	Logistics	New orders arrive and one van breaks down mid-route; the planner must re-route live, not just solve a static benchmark	Solomon VRPTW benchmark (SINTEF)

Sources

- Bitext Customer Support dataset — Hugging Face
- Online Retail II — UCI Machine Learning Repository
- MaintNet: open-source library for predictive maintenance language resources (COLING 2020)
- AI4I 2020 Predictive Maintenance Dataset — UCI Machine Learning Repository
- AI4I 2020 class-imbalance analysis (3.39% failures)
- PlantDoc Dataset — GitHub
- NEU Surface Defect Database — Northeastern University
- Air Quality Data in India (2015–2020) — Kaggle
- ACN-Data — Caltech and ACN-Data registration terms
- ACN-Data static copy — GitHub
- buildingSMART Sample-Test-Files — GitHub
- IfcOpenShell installation docs
- Solomon VRPTW benchmark — SINTEF
- Vehicle Routing Problem with Time Windows — Google OR-Tools

PS 1.1 · Code-Mixed Support Triage Assistant

Track 1 · Customer Service · Difficulty ★ · Skills: LLM prompting, text classification, web app

1. Problem:

Indian customers rarely write to support in textbook English; "order vandhuchu but box damaged, refund venum" is a normal message. Support teams lose hours reading and routing these by hand. Build a triage assistant that reads an incoming message, labels its intent and urgency, drafts a reply, and routes it to the right queue.

Core challenge: the assistant must handle Tanglish/Hinglish it was never trained on, and hand off to a human with a stated reason when it is unsure, instead of guessing.

2. What to Build

Must-have (MVP):

1. An intent and urgency classifier over the dataset's categories (orders, refunds, accounts, etc.) that returns a confidence score.
2. A team-written test set of at least 30 code-mixed messages (Tanglish and/or Hinglish) with expected labels, and the classifier's results on it shown in the app.
3. A deployed inbox view: submit a message and get intent, urgency, a draft reply, and an "escalate to human" flag with a one-line reason when confidence is below your threshold.

Stretch goals:

- Draft the reply in the customer's own language mix.
- A queue dashboard showing volume by intent and urgency.
- Pick few-shot examples by similarity to the incoming message.

3. Data & Resources

- **Data:** Bitext Customer Support dataset, Hugging Face · CDLA-Sharing-1.0 · about 27,000 English request/response pairs labelled with category and intent · use the full file · load with `datasets.load_dataset("bitext/Bitext-customer-support-llm-chatbot-training-dataset")`.
- **Suggested stack:** an LLM with a free tier (e.g. Gemini or Groq) or a small open model; a TF-IDF + logistic regression baseline in scikit-learn for comparison; Streamlit, FastAPI or Next.js; deploy to Vercel or Hugging Face Spaces.
- **Watch-outs:**
  - The dataset is synthetic English with placeholders like `{{Order Number}}`; clean them before use.
  - Keep your code-mixed test set out of any few-shot prompt, or your results are meaningless.
  - Call the LLM from a backend; never ship an API key in frontend code.

4. Evaluation Metrics (optional)

- Macro-F1 on a held-out Bitext split and, separately, on your code-mixed test set.
- Escalation rate, and how many escalated messages were actually wrong predictions.

PS 1.2 · Ask-Your-Data Sales Agent (text-to-SQL)

Track 1 · Retail · Difficulty ★★ · Skills: SQL, LLM tool calling, data cleaning, web app

1. Problem

Store managers have questions ("which products were returned most last quarter?") but no time to learn SQL, and analysts are a bottleneck. Build an agent that answers plain-English questions over two years of real online retail transactions by writing and running SQL, then showing the result as a table or chart.

Core challenge: real retail data is messy. Cancelled invoices, returns recorded as negative quantities, missing customer IDs and non-product codes (postage, manual adjustments) all produce confidently wrong answers if ignored. The agent must get these right, show its SQL, and ask back when a question is ambiguous ("best customers" by revenue or by order count?).

2. What to Build

Must-have (MVP):

1. The data loaded into DuckDB or SQLite, with a short written "data rules" note in the README covering how cancellations, returns and missing customers are treated.
2. An agent that turns a question into SQL, runs it on a read-only connection, retries if the SQL fails, and returns the answer with the SQL shown.
3. A trap-question suite of at least 10 team-written questions (5 or more involving cancellations, returns or ambiguity) with expected answers from hand-written SQL; the deployed app runs the suite and shows pass/fail.

Stretch goals:

- A clarifying-question step for ambiguous questions.
- An automatic chart choice (bar vs line) based on the result shape.
- Multi-turn follow-ups ("now only for Germany").

3. Data & Resources

- **Data:** Online Retail II, UCI · CC BY 4.0 · 1,067,371 transaction rows from a UK online gift retailer, Dec 2009 to Dec 2011, about 44 MB · use the full file or just the 2010–11 sheet · download the Excel workbook from the UCI page (or `fetch_ucirepo(id=502)`) and convert it once to Parquet or a DuckDB file.
- **Suggested stack:** DuckDB, an LLM with a free tier, LangGraph or a hand-rolled tool loop, Streamlit or Next.js, deployed to Hugging Face Spaces or Render.
- **Watch-outs:**
  - An invoice number starting with "C" means a cancellation; many customers are wholesalers, so a few accounts dominate revenue.
  - Prices are in pounds sterling; say so in answers.
  - Never give the LLM write access. Block anything that is not a `SELECT`.

4. Evaluation Metrics (optional)

- Execution accuracy on the trap suite: share of questions whose result matches the expected answer.
- Share of ambiguous questions where the agent asked back instead of guessing.

PS 1.3 · Maintenance Copilot: Agentic RAG over Logbooks

Track 1 · Aviation Maintenance · Difficulty ★★★ · Skills: retrieval (RAG), LLM agents and tools, text normalisation, web app

1. Problem

When a technician sees a fault, the fastest fix is often buried in years of past logbook entries: what was reported, and what action cleared it. Build a copilot that takes a fault description, finds similar past records, summarises the likely causes and the actions that fixed them, and shows its evidence.

Core challenge: logbook text is terse, abbreviation-heavy and misspelled, so plain embedding search misses obvious matches. Every answer must cite the record IDs it relied on, and the copilot must say "insufficient evidence" rather than invent a procedure. In maintenance, a confident wrong answer is worse than none.

2. What to Build

Must-have (MVP):

1. An ingestion step that normalises the text (expanding abbreviations with MaintNet's abbreviation list or your own) and builds a hybrid index: keyword (BM25) plus embeddings.
2. An agent with at least two tools (e.g. `search_records`, `get_record`) that answers a fault query with cited record IDs, and refuses when the best matches are weak.
3. A deployed chat UI that shows the answer, the cited records side by side, and the agent's tool-call trace.

Stretch goals:

- A recurring-defect detector that flags issues reported again and again.
- Also index MaintNet's facility-maintenance records and let the user pick the domain.
- A query-rewrite step that expands the technician's own shorthand before searching.

3. Data & Resources

- **Data:** MaintNet aviation maintenance logbook · open research library; check the MaintNet page for terms and cite the COLING 2020 paper · 6,169 records (about 77,000 tokens) of free-text problems and the actions taken, from the University of North Dakota Aviation Program · use the aviation set · download the CSV from the MaintNet page (a Kaggle mirror, "Aircraft Historical Maintenance Dataset (2012–2017)", also exists).
- **Suggested stack:** LangGraph or LlamaIndex; `rank_bm25` plus a small sentence-transformers model; FAISS, Chroma or LanceDB; an LLM with a free tier; Streamlit or Next.js.
- **Watch-outs:**
  - Records are short, so similarity scores bunch together; set and justify a refusal threshold.
  - Show a clear disclaimer that answers are not approved maintenance procedures.
  - Build your test questions before tuning, or you will tune to them.

4. Evaluation Metrics (optional)

- Retrieval hit@5 on 15 team-written question-to-record pairs.
- Citation precision: share of cited records that actually support the answer.
- Correct refusals on 5 questions the data cannot answer.

PS 2.1 · Maintenance Shift Planner

Track 2 · Industrial · Difficulty ★ · Skills: tabular ML, cost modelling, simple scheduling, web app

1. Problem

Unplanned downtime on a production line costs far more than scheduled maintenance, but a plant has only a few technicians per shift and every false alarm wastes their time. Build a deployed tool that predicts which machines are likely to fail, sets the alert threshold from an explicit cost trade-off (missed failure vs false alarm), and then assigns the shift's limited technician slots to the machines that matter most. Predicting failure is only the input; the deliverable is the shift plan.

Core challenge: with only a few technicians per shift, decide which machines get serviced and justify it with cost, not accuracy.

2. What to Build

Must-have (MVP):

1. A model plus a `/predict` endpoint (or Streamlit form) that takes one machine reading and returns failure probability and predicted failure mode.
2. An alert threshold chosen from a stated cost model (e.g. missed failure = 20× a false alarm), with the cost-vs-threshold curve shown.
3. A publicly deployed shift planner: replay a batch of readings as one shift, take the number of technician slots as input (e.g. 3), and show which machines get serviced and why.

Stretch goals:

- Top-3 contributing features for each prediction (SHAP).
- Per-machine history view.
- Drift detection that flags when incoming telemetry stops looking like the training data.
- An LLM-drafted maintenance ticket built from the prediction's explanation.

3. Data & Resources

- **Dataset:** AI4I 2020 Predictive Maintenance, UCI · CC BY 4.0 · 10,000 rows, 14 columns (air and process temperature, rotational speed, torque, tool wear, a failure flag and five failure-mode labels) · use the full file · load with `pip install ucimlrepo`, then `fetch_ucirepo(id=601)`.
- **Suggested stack:** pandas, scikit-learn or XGBoost, SHAP, FastAPI, Streamlit or Next.js; deploy to Hugging Face Spaces, Render or Vercel.
- **Watch-outs:**
  - Only about 3.4% of rows are failures, so accuracy is meaningless. Use class weights or resampling.
  - Drop `UID` and `Product ID`; they leak row identity and teach nothing.
  - The data is synthetic. Say so in the README and the pitch.

4. Evaluation Metrics (optional)

- PR-AUC on a held-out split.
- Recall at the precision your cost model selects.

PS 2.2 · Leaf Doctor for Real Field Photos

Track 2 · Agriculture · Difficulty ★ · Skills: computer vision, transfer learning, image quality checks, web app

1. Problem

Plant diseases cost Indian farmers a large share of their yield, and early diagnosis usually needs an expert who isn't nearby. Most disease classifiers are trained on clean lab photos and fail on real phone pictures taken in a field. Build an app that diagnoses a leaf from a real-world photo and is honest about what it doesn't know.

Core challenge: a confident wrong diagnosis can lead to the wrong pesticide. The app must reject unusable photos with specific retake advice, and answer "not sure" below a confidence bar instead of always naming a disease.

2. What to Build

Must-have (MVP):

1. A transfer-learning classifier (e.g. MobileNetV3 or EfficientNet-B0 pretrained on ImageNet) trained on PlantDoc, restricted to 3–5 species if time is short.
2. A photo-quality gate that checks blur, brightness and whether the leaf fills the frame, and rejects poor photos with a specific tip ("move closer", "avoid shadow").
3. A deployed, mobile-friendly app: upload or capture a photo, pass the quality check, see the top-3 predictions with confidence, or a "not sure, please consult an expert" message below your threshold.

Stretch goals:

- A Grad-CAM heatmap showing which part of the leaf drove the prediction.
- A Tamil or Hindi interface.
- A short care note per disease, cited from an agricultural extension source.

3. Data & Resources

- **Data:** PlantDoc dataset, GitHub · CC BY 4.0 · 2,598 internet-sourced field images across 13 plant species and up to 17 disease classes, published by researchers at IIT · use 3–5 species for a 7-hour build · clone the repository; images are organised in class folders.
- **Suggested stack:** PyTorch with timm, or TensorFlow/Keras; OpenCV for the quality checks (variance of the Laplacian for blur); Gradio or Streamlit on Hugging Face Spaces.
- **Watch-outs:**
  - Classes are small and imbalanced; use augmentation and report per-class results.
  - Don't start from a PlantVillage-trained model; that is the lab-photo problem this statement avoids.
  - Internet-scraped images can repeat; check that no test image also sits in training.

4. Evaluation Metrics (optional)

- Top-1 and top-3 accuracy on the PlantDoc test split.
- Accuracy on accepted images vs the share of images rejected or marked "not sure" (a coverage curve).

PS 2.3 · Unknown-Defect Steel Inspector

Track 2 · Manufacturing QA · Difficulty ★★ · Skills: computer vision, embeddings, open-set recognition, web app

1. Problem

Visual inspection models on steel lines are trained on the defects seen so far. When a new kind of defect appears, a normal classifier forces it into one of the known classes, and bad steel ships with the wrong label. Build an inspector that classifies known surface defects, recognises when it is looking at something new, and learns the new defect from a handful of operator-labelled examples.

Core challenge: one defect class is held out of training entirely. The system must flag those images as "unknown" instead of mislabelling them, then add the class from just 5 labelled examples without full retraining.

2. What to Build

Must-have (MVP):

1. A classifier trained on 5 of the 6 defect classes; state in the README which class you held out.
2. An unknown-detection method (confidence threshold, energy score, or distance to class centroids in embedding space) that flags held-out-class images, shown in a confusion matrix that includes "unknown".
3. A deployed inspector UI: upload an image and get a defect class or "unknown, send to operator"; the operator labels 5 examples of the new defect, the system adds it as a class (e.g. a new prototype in embedding space), and it is recognised from then on.

Stretch goals:

- Bounding boxes using the detection version of the dataset (NEU-DET).
- An inspection log showing defect counts per shift.
- Let the operator reject or merge a newly added class.

3. Data & Resources

- **Data:** NEU Surface Defect Database, Northeastern University · free for research; cite Song & Yan (2013) · 1,800 grayscale 200×200 images of hot-rolled steel strip, 300 each of six defects: crazing, inclusion, patches, pitted surface, rolled-in scale, scratches · download the classification set, NEU-CLS (not NEU-DET).
- **Suggested stack:** PyTorch with timm (ResNet-18 or EfficientNet-B0), scikit-learn for centroid distances and AUROC, Gradio or Streamlit on Hugging Face Spaces.
- **Watch-outs:**
  - Images are grayscale; repeat the channel to feed an ImageNet-pretrained model.
  - Known-class accuracy will look very high. The real test is unknown detection, so report it separately.
  - Keep the 5 "operator" examples out of every evaluation set.

4. Evaluation Metrics (optional)

- Accuracy on the 5 known classes.
- AUROC for known vs unknown images.
- Accuracy on the new class after the 5-example update, and whether known-class accuracy dropped.

PS 3.1 · Safe Outdoor Hours Planner for Schools

Track 3 · Environment · Difficulty ★ · Skills: time-series analysis, data-quality checks, data visualisation, web app

1. Problem

Schools schedule sports, assemblies and outdoor classes without knowing when the air is safest, even though pollution in Indian cities swings sharply through the day. Build a planner that tells a teacher which hours are safest for outdoor activity in their city, in plain language.

Core challenge: this is not AQI prediction. The planner recommends time windows from hourly data, and it must refuse to advise when the readings behind a recommendation are missing, stale or stuck, rather than showing a reassuring number it can't back up.

2. What to Build

Must-have (MVP):

1. A data-quality layer that marks each city-hour as reliable or not: missing values, stale readings, or a sensor stuck at the same value for hours.
2. A planner that takes a city, a date and an activity length (e.g. 60 minutes) and returns the best window between 6 am and 6 pm using the CPCB AQI bands, with reasons, or "no advice" when the data is unreliable.
3. A deployed app showing an hour-by-hour strip coloured by AQI band, the recommended window, and a one-paragraph advisory a teacher can forward to parents.

Stretch goals:

- A typical-day heatmap by month, so schools can plan a term ahead.
- Different thresholds for sensitive groups (e.g. students with asthma).
- Compare two cities side by side.

3. Data & Resources

- **Data:** Air Quality Data in India (2015–2020), Kaggle · data originally from the Central Pollution Control Board; check the Kaggle page for licence terms · hourly and daily pollutant readings and AQI for 26 cities including Bengaluru, Chennai, Coimbatore and Delhi · use `city_hour.csv` (about 16 MB) for 2–3 cities · download from Kaggle (free account).
- **Suggested stack:** pandas, Plotly or Altair, Streamlit or Next.js, deployed to Hugging Face Spaces or Vercel.
- **Watch-outs:**
  - The data ends in 2020, so the app runs in replay mode on historical dates; say so in the UI.
  - Early years have many gaps; your reliability rules should handle them, not drop them silently.
  - Use the official National AQI bands (Good, Satisfactory, Moderate, Poor, Very Poor, Severe) and cite CPCB.

4. Evaluation Metrics (optional)

- Back-test: how often the recommended window matches the window with the lowest observed AQI that day.
- Share of city-hours flagged unreliable, by city and year.

PS 3.2 · Workplace EV Charging Scheduler

Track 3 · Clean Mobility · Difficulty ★ · Skills: simulation, scheduling algorithms, data visualisation, web app

1. Problem

At an office or campus, most EVs plug in at the same time each morning. Charging them all at full power creates a sharp peak that the site's electrical connection may not handle, and upgrading it is expensive. Build a scheduler that shares a limited power budget across parked cars so drivers still get the energy they need before they leave.

Core challenge: replay real charging sessions and keep total site load under a power cap you set, while meeting each driver's energy need by their departure time as far as possible. Prove it by comparing against uncontrolled charging on the same data.

2. What to Build

Must-have (MVP):

1. A replay simulator that steps through one week of sessions from one site in 15-minute intervals.
2. Two policies: uncontrolled (every car charges at full rate on arrival) and your scheduler (e.g. earliest-departure-first or least-laxity-first under the cap).
3. A deployed dashboard: pick a day and a cap, then see site load for both policies against the cap line, plus peak kW, share of energy delivered, and how many drivers left short.

Stretch goals:

- Solar-aware charging that shifts load toward midday.
- Cost under a time-of-use tariff.
- A driver view: "you will get about X kWh by your departure time".

3. Data & Resources

- **Data:** ACN-Data, Caltech · free registration; educational and research use only · tens of thousands of real workplace charging sessions from Caltech, JPL and an office site, with plug-in time, unplug time, energy delivered and (for some sessions) the driver's requested energy and departure time · use one week at one site · download through the `acnportal` Python client with your API token, or use the static copy on GitHub (2018–2020, no sign-up).
- **Suggested stack:** pandas and NumPy for the simulator, Plotly for charts, Streamlit on Hugging Face Spaces.
- **Watch-outs:**
  - Register and download before 22 Oct; don't rely on sign-up working on the day.
  - Many sessions lack driver inputs; use actual energy delivered and unplug time as stand-ins and say so.
  - `acnportal` ships reference algorithms. You may compare against them, but the scheduler you present must be your own.

4. Evaluation Metrics (optional)

- Peak kW reduction vs uncontrolled charging.
- Share of needed energy delivered, and number of sessions under-served.

PS 3.3 · BIM Model Checker (IFC)

Track 3 · Construction · Difficulty ★★ · Skills: file-format parsing, rules engines, Python backend, web app

1. Problem

Building designs are exchanged as BIM models in the open IFC format, and errors found late on site are expensive. A model is also useless for energy or carbon analysis if key data is missing: windows without thermal values, walls without materials. Build a checker that reads an IFC model and reports what is wrong or missing before it reaches site.

Core challenge: you design the rule set. Write at least 5 rules, at least 2 of them on energy or sustainability data completeness, and report every violation with the element's ID and location so a designer can fix it.

2. What to Build

Must-have (MVP):

1. A parser that loads an IFC file and lists storeys, spaces, doors, windows and walls with their key properties.
2. A rules engine driven by a config file (YAML or JSON) with at least 5 rules, e.g. doors narrower than 900 mm, unnamed spaces, windows with no thermal transmittance (U-value), walls with no material, storeys with no spaces.
3. A deployed web app: upload an IFC file and get a summary plus a violation table (rule, element type, GlobalId, storey, value found), downloadable as CSV.

Stretch goals:

- A 3D viewer that highlights violating elements.
- An embodied-carbon estimate from wall and slab quantities, using emission factors cited from a public source.
- Import rules from an IDS (Information Delivery Specification) file instead of your own config.

3. Data & Resources

- **Data:** buildingSMART Sample-Test-Files, GitHub · CC BY 4.0 · official sample models across IFC schema versions · pick 2–3 complete building models · clone with Git LFS installed or download single files from GitHub. Backup source: the Open IFC Model Repository linked in that repo's README.
- **Suggested stack:** Python with IfcOpenShell (`pip install ifcopenshell`, then `ifcopenshell.open("model.ifc")`), FastAPI or Streamlit; for the 3D stretch goal, web-ifc / That Open Engine (built on Three.js).
- **Watch-outs:**
  - Read the project's length units first; millimetres vs metres breaks every size rule.
  - IFC2x3 and IFC4 store some properties differently; test on both.
  - A missing property set is itself a finding, so report it rather than crash.

4. Evaluation Metrics (optional)

- Deliberately break a clean model with 10 known violations, then report how many your checker found and how many false alarms it raised.

PS 4.1 · Markdown Before Waste: Near-Expiry Pricing

Track 4 · Grocery Retail · Difficulty ★ · Skills: simulation, pricing rules, data visualisation, web app

1. Problem

Grocery stores throw away perishables (bread, milk, fresh produce) that pass their expiry date unsold. Discounting earlier sells more of them, but discounting too much or too soon eats the margin. Build a tool that tests markdown rules on a simulated store and shows the trade-off between waste and profit.

Core challenge: there is no ready dataset, so you design the store simulator yourself. Every assumption (shelf life, demand by weekday, how much a discount lifts sales) must be written down and justified, and your results must hold up when those assumptions change.

2. What to Build

Must-have (MVP):

1. A simulator for one store with about 20 perishable products over 30 days: stock arrives in batches with expiry dates, and daily demand depends on price and days to expiry. Document every assumption in the README.
2. Two policies: fixed price, and your markdown rule (e.g. 20% off with 2 days left, 40% with 1 day left) with adjustable thresholds.
3. A deployed dashboard: run both policies on the same random seed and compare units wasted, revenue and margin, plus a per-product view of what was marked down and when.

Stretch goals:

- Search for the best discount steps per product category.
- A sensitivity panel: what happens if shoppers respond half as strongly to discounts?
- A shopper-facing "today's deals" page generated from the simulation.

3. Data & Resources

- **Data:** synthetic, generated by your simulator. Minimum fields per product: category, shelf life in days, unit cost, base price, average daily demand, and a price-sensitivity factor. Optionally, take the weekday shape of demand from real sales data (e.g. Online Retail II, UCI) and cite it.
- **Suggested stack:** Python with NumPy and pandas, or a JavaScript simulation that runs in the browser; Streamlit or Next.js on Vercel or Hugging Face Spaces.
- **Watch-outs:**
    - Fix and display the random seed so judges can reproduce your numbers.
    - A rule that discounts everything always cuts waste; show margin next to waste so that doesn't look like a win.
    - Keep the simulator simple enough to explain in one slide.

4. Evaluation Metrics (optional)

- Waste reduction (%) and margin change vs fixed price, averaged over at least 10 seeds.

PS 4.2 · Warehouse Digital Twin: 3D Picking Floor Game

Track 4 · Warehousing / Digital Twin · Difficulty ★★ · Skills: 3D web graphics (Three.js), simulation, data pre-processing, game design

1. Problem : In a warehouse, pickers can spend more time walking between shelves than picking. Where each product sits (its "slot") decides how far they walk. Build a browser-based 3D digital twin of a picking floor, driven by real orders, and turn slotting into a game: the player re-arranges products and the twin shows how much walking it saves.

Core challenge: the twin must be driven by real order data, not a static 3D model. The player's layout is scored by replaying exactly the same orders before and after, so the improvement is measured, not claimed.

2. What to Build

Must-have (MVP):

1. A pre-processing step that turns one month of order lines into JSON: the top ~100 products, how often each is picked, and the list of orders.
2. A 3D scene of a grid warehouse (racks, aisles, a packing dock and at least one animated picker) that replays orders: the picker walks along aisles to each product's slot and back, with live distance and time counters.
3. A game mode: the player swaps or drags products between slots, replays the same orders, and sees before/after walking distance and pick time. Include an automatic baseline to beat (fast movers nearest the dock).

Stretch goals:

- Several pickers, with congestion when two meet in an aisle.
- Batch picking: one trip serves several orders.
- A heatmap of rack visits, plus a leaderboard of player layouts.

3. Data & Resources

- **Data:** Online Retail II, UCI · CC BY 4.0 · real order lines (invoice, product code, quantity, timestamp) · use one month · download the Excel workbook from UCI and pre-process it with pandas. The warehouse layout (rack count, aisle widths, walking speed) is yours to design; state it in the README.
- **Suggested stack:** Three.js or React Three Fiber with `@react-three/drei`; free CC0 low-poly assets (e.g. Kenney) or plain boxes; Python for pre-processing; deploy as a static site on Vercel or Cloudflare Pages.
- **Watch-outs:**
  - Remove cancelled invoices (numbers starting with "C") and non-product codes before counting picks.
  - Walking distance must follow aisles, not straight lines through racks.
  - Keep geometry simple so the scene runs smoothly on an average laptop during the demo.

4. Evaluation Metrics (optional)

- Total walking distance and average pick time per order, for the original layout, the automatic baseline and the player's layout.

PS 4.3 · Last-Mile Route Planner with Live Re-planning

Track 4 · Logistics · Difficulty ★★★ · Skills: combinatorial optimisation, event simulation, backend APIs, map visualisation

1. Problem

Delivery fleets plan routes in the morning, but the day never goes to plan: new orders come in, customers have delivery windows, and vans break down. Build a dispatcher tool that plans routes for a fleet with capacity and time-window limits, and re-plans live when the day changes.

Core challenge: solving a static benchmark is not enough. Mid-day, new orders arrive and one van breaks down; the planner must re-route from the fleet's current state (completed stops fixed, vans starting where they are) within seconds, and show what the disruption cost.

2. What to Build

Must-have (MVP):

1. A static solver: load a Solomon instance (start with a 25-customer one such as C101 or R101), solve it with capacity and time windows, and report vehicles used and total distance against the best known result published by SINTEF.
2. An event simulator with a clock that can inject (a) 5 new orders at a chosen time and (b) a breakdown of one vehicle, then re-plan from the current state, reassigning the broken van's remaining stops.
3. A deployed dispatcher dashboard: routes drawn on a 2D map, a timeline slider, event buttons, and before/after KPIs (distance, late deliveries, orders not served).

Stretch goals:

- Scale to the 100-customer instances.
- Compare full re-solve vs a quick local repair on time and route quality.
- A driver-facing mobile view showing the next stop and ETA.

3. Data & Resources

- **Data:** Solomon VRPTW benchmark, SINTEF · public academic benchmark (1987) · instances with 25, 50 and 100 customers in clustered (C), random (R) and mixed (RC) layouts, with best known solutions listed · start with 25-customer instances · download the plain-text instance files from the SINTEF pages.
- **Suggested stack:** Python with Google OR-Tools (`pip install ortools`; see the OR-Tools VRPTW guide), FastAPI, Leaflet or Plotly for routes, React or Streamlit; deploy on Render or Hugging Face Spaces (OR-Tools is too heavy for edge functions).
- **Watch-outs:**
  - OR-Tools needs integer costs; scale distances (e.g. ×100) before rounding.
  - Set a solver time limit (5–10 s) so re-planning stays interactive in the demo.
  - Copying the OR-Tools VRPTW sample covers only part of item 1; the re-planning is what this problem is about.

4. Evaluation Metrics (optional)

- Static: total distance and vehicles vs the best known solution.
- Dynamic: re-plan time, late deliveries and orders dropped after each event.